

★ PRIMER · UNIT 12

Before the Lecture: Turning Words Into Numbers

Unit 12 · NLP, LLMs and RAG

Watch this first if vectors, cosine and softmax are hazy

Twenty minutes of groundwork — how a sentence becomes a list of numbers, how you measure whether two meanings are close, and what an LLM is really computing.

How to Use This Primer

Two audiences, one deck — read the column that describes you

If this is new to you

- Watch before the lecture.
- Cosine and softmax are here.
- Do slide 8's arithmetic.

If you have met it before

- Skim it — this is revision.
- Slide 7 is what finance changes.
- Slide 16 says if you can skip.

Nothing here is examined on its own — it is the floor the lecture stands on

Where Unit 12 Sits

The course builds one pipeline; here is this unit's stone

Unit 11

done

Building portfolios that survive.

Unit 12

you are here

Machines that read text.

Unit 13

next

Learning to act —
reinforcement.

Every filing, transcript and news wire is data. This unit is how a model reads it.

Why Text Must Become Numbers

A model cannot see a word. It can only see numbers — so meaning must first be given coordinates.

The old way

- Count how often words appear.
- Order is thrown away.
- 'not good' looks positive.

The modern way

- Each word gets a position.
- Similar meanings sit close.
- Order is kept.

An embedding is that position — a list of numbers standing in for a meaning.

The Words the Lecture Assumes

Five terms, and both halves of the unit open up

Word	What it means	Here
Token	a chunk of text	a word, or part of one
Embedding	a list of numbers for meaning	a point on a map of ideas
Cosine similarity	the angle between two vectors	how close two meanings are
Softmax	turns scores into probabilities	which word comes next
RAG	retrieve, then generate	look it up before answering

Cosine ignores length and looks only at direction — which is exactly what meaning needs.

How a Chatbot Grew Up

The lecture traces all four stages, and finance used every one

Stage	How it answers	Limit
Rules	a decision tree of replies	brittle, endless to maintain
TF-IDF + SVM	classify the intent	no memory, no nuance
BERT	understands the sentence	cannot generate freely
LLM + RAG	retrieves, then writes	confident when it is wrong

Each stage kept the previous one's job and added a new one. None of them replaced it.

What an LLM Is Actually Doing

It is predicting the next token, over and over. Everything else you see is that loop, run fast.

One step

- Score every possible token.
- Softmax into probabilities.
- Pick one, append it.

Why it feels like thought

- Repeat a thousand times.
- Each step sees all before it.
- Coherence emerges.

Nothing in that loop checks whether the sentence is true. That is what RAG is for.

Cosine Similarity by Hand

Two three-dimensional meanings — how close are they?

Step	Working	Value
Vectors	$a = (1, 2, 2), b = (2, 0, 4)$	—
Dot product	$1 \times 2 + 2 \times 0 + 2 \times 4$	10
Length of a	root of $1 + 4 + 4$	3
Length of b	root of $4 + 0 + 16$	4.47
Cosine	$10 / (3 \times 4.47)$	0.75

0.75 is fairly close. A cosine of 1 means identical direction; 0 means unrelated.

One Next-Word Choice

Three scores through softmax — this is every LLM step

The scores

- 'rose' scores 2.0.
- 'fell' scores 1.0.
- 'banana' scores 0.0.

After softmax

- rose: 0.67.
- fell: 0.24.
- banana: 0.09.

Temperature stretches or flattens those three numbers — that is the whole personality dial

Why RAG Exists

An LLM knows what it was trained on, and nothing after that

Without retrieval

- It answers from memory.
- Memory has a cut-off date.
- Gaps are filled confidently.

With retrieval

- Search your documents first.
- Paste the passage into the prompt.
- Now it answers from evidence.

In finance this is not optional. An answer without a citable source is not usable.

How RAG Works

Three moves, and the middle one is where cosine earns its keep

1 · Embed

the question

Turn the user's question into a vector of numbers.

2 · Retrieve

by cosine

Find the document chunks whose vectors point the same way.

3 · Generate

with the evidence

Give the model the question and the passages, then let it write.

The retrieval step is the hand calculation you just did, run against a million chunks.

Three Ways Text Becomes Numbers

The lecture uses all three, in this order

Method	What it captures	Blind to
Bag of words	which words appear	order, and meaning
TF-IDF	which words are distinctive	order, and synonyms
Embeddings	meaning and context	nothing much — but costly

FinBERT is an embedding model trained on financial text, so 'liability' reads correctly.

Notation You Will Meet

Four symbols carry the whole unit

Symbol	Read it as	It stands for
$a \cdot b$	a dot b	the dot product, again
$\ a \ $	norm of a	the length of a vector
$\cos \theta$	cos theta	similarity — the angle between meanings
T	temperature	how adventurous the next-token pick is

The dot product returns for the third time. It is the workhorse of this whole course.

The Python You Will See

You do not need to write it yet — you need to recognise it

Line	What it does
<code>TfidfVectorizer().fit_transform(docs)</code>	the classical text-to-number step
<code>cosine_similarity(a, b)</code>	the hand calculation you just did
<code>model.encode(sentences)</code>	turn sentences into embeddings
<code>np.argsort(sims)[-5:]</code>	the five most similar chunks — retrieval
<code>softmax(scores / T)</code>	next-token probabilities, with temperature

A working RAG prototype is these five lines plus a document loader. Genuinely.

Three Things Beginners Get Wrong

Each one costs a week if you carry it into unit 13

1

"The model understands the sentence"

It predicts likely continuations. That produces understanding-shaped output, which is not the same.

2

"A confident answer is a correct answer"

Confidence and accuracy are unrelated in a language model. This is what RAG is meant to fix.

3

"Longer text means a bigger cosine"

Cosine divides by length precisely so it does not. That is why it is used and not the dot product.

If one of these surprised you, that is exactly what the primer was for

Self-Check Before the Lecture

Answer these three out loud — if you can, you are ready

1 Dot product 10, lengths 3 and 4.47. What is the cosine?

0.75

2 What turns three scores into three probabilities?

softmax

3 What does the R in RAG add?

retrieval

Cover the right column, answer, then check. All three come from this deck.

What the Lecture Does With This

You bring the vocabulary; the lecture brings the argument

1

It maps meaning to numbers

One sentence's journey into a vector, and cosine similarity worked by hand.

2

It opens the language model

What an LLM computes at each step, and what temperature does to that choice.

3

It ends in code

The problem set measures meaning, runs softmax, and builds a small finance chatbot.

Next: Unit 12 lecture — Teaching Machines to Read